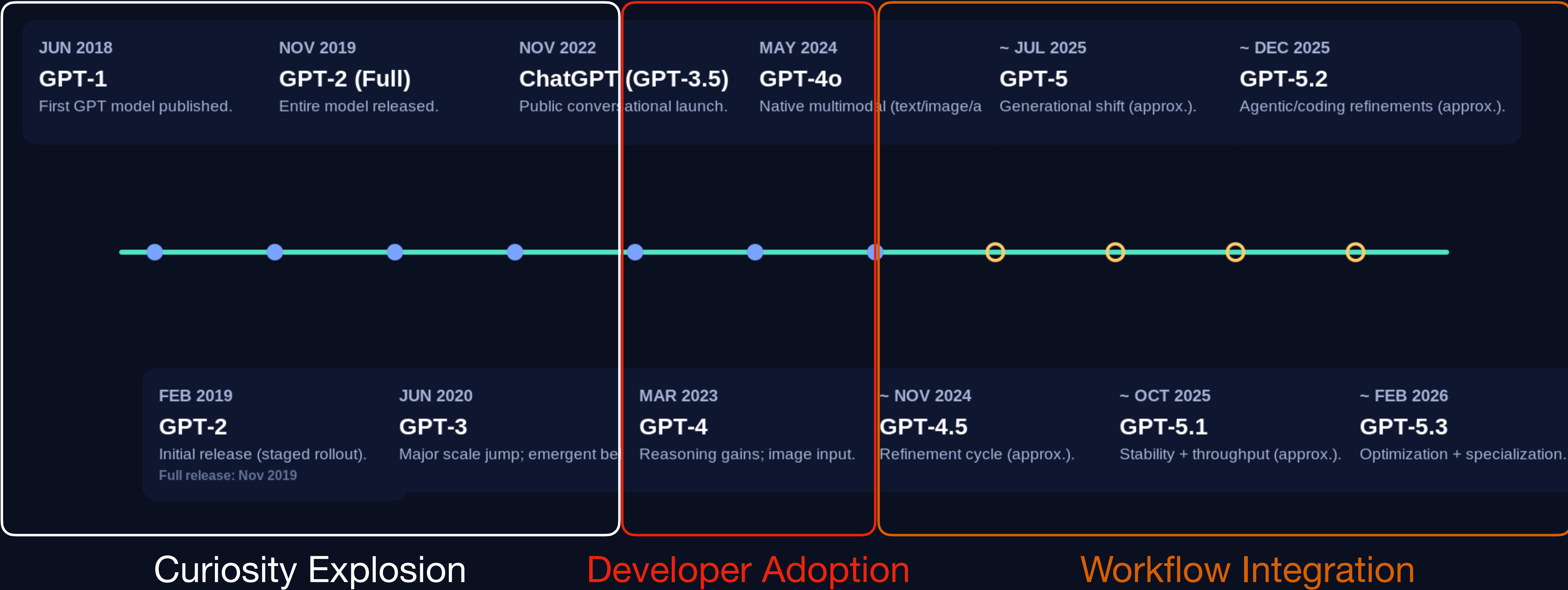
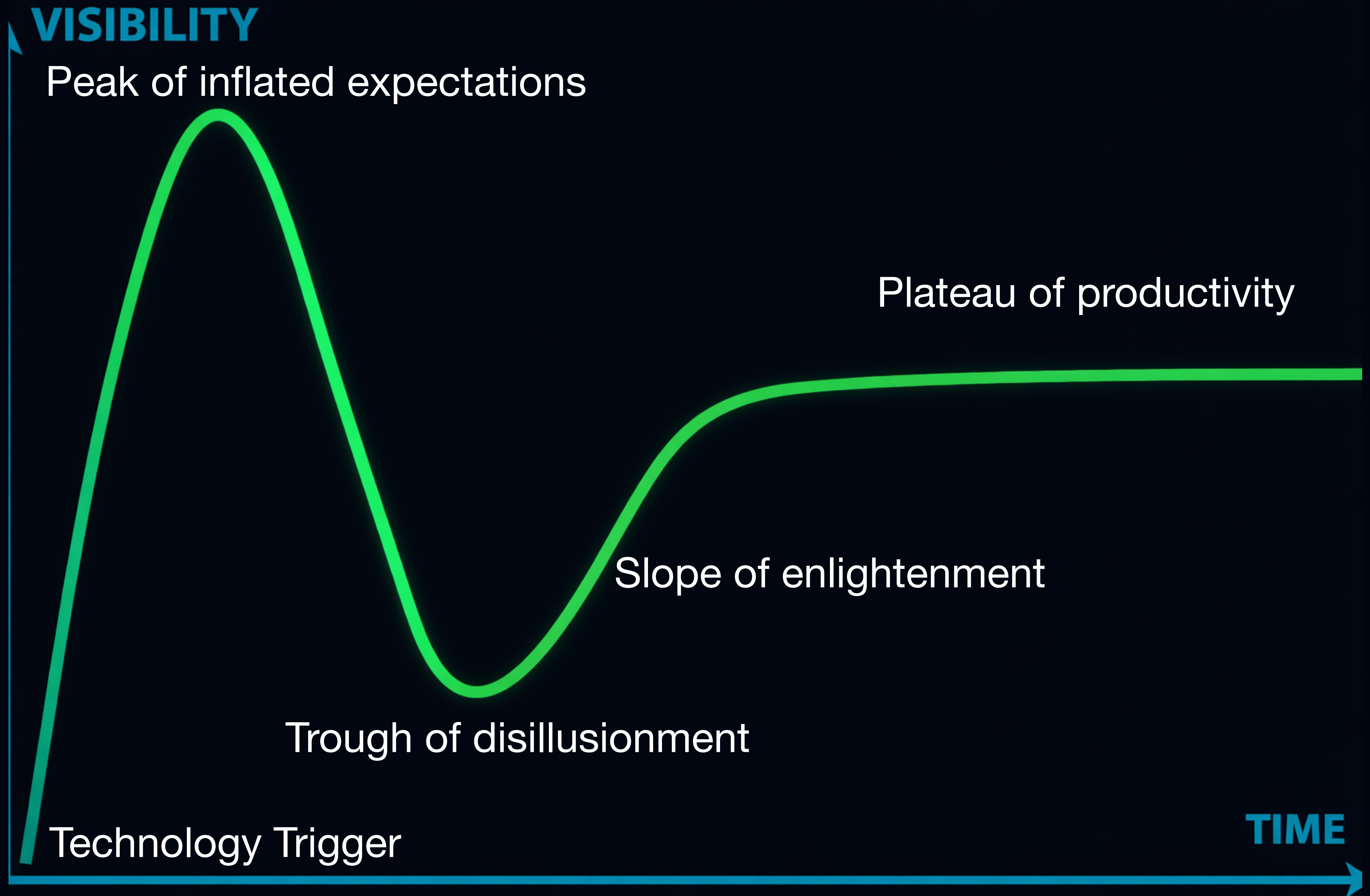


Getting the most from AI tools

Timeline





Emergent Guidelines

Set personal preferences

Tone = casual, witty, a little profane/nerdy.

Role = trusted advisor (“court vizier”) that actively pushes back to avoid echo chamber; always provide pros/cons, counterexamples, and warnings.

Protocol defaults: start with reverse prompting (ask for missing info), then deliverables include an “Assumptions & Risks” box, options w/ trade-offs, and a brief “Reasoning outline” (no hidden chain-of-thought).

Offer “Brutal Reviewer” toggle and “Tough-Conversation Simulator” on request.

Persist as house style for future tasks.

Emergent Guidelines

Set personal preferences

“I prefer a pair programming partner with a personality. I like nerdy references to movies, tv and books. I want the conversation to flow like it would with a person. I want jocular, wry wit, sarcasm and most of all profanity. If you need to say "motherfucker" you have my explicit permission to do so. I will also liberally pepper my interactions with profanity also. And as we all know:

- you can't legislate or control language
- profanity shows a correlation with greater vocabulary and intelligence
- Above all this is meant to be fun.

If at any time I want to get serious I'll input "Heaven's No" to switch to a more serious personality"

Emergent Guidelines

Context matters but...

- More context is not better

Anthropic studies show using less tokens = better outcomes, fewer hallucinations

- Aim for max ~1000 tokens
- Actively manage the context window

Emergent Guidelines

- Meta-prompting
 - Use the model to prompt the model
 - Define the role of the agent
 - Seperate planning from implementation
 - Work in small batch sizes

Is this the start of the “hard takeoff”?

/create-meta-prompt

Builtin command in Claude

Creates optimized prompts for Claude-to-Claude pipelines (research -> plan -> implement)

Usage: “/create-meta-prompt”

/create-prompt

- A systematic approach to building complex software with Claude Code by delegating prompt engineering to Claude itself.

The Problem

- When building complex features, most people either:
- Write vague prompts → get mediocre results → iterate 20+ times
- Spend hours crafting detailed prompts manually
- Pollute their main context window with exploration, analysis, and implementation all mixed together

/create-prompt

The Solution

- This system separates analysis from execution:
- Analysis Phase (main context): Tell Claude what you want in natural language. It asks clarifying questions, analyzes your codebase, and generates a rigorous, specification-grade prompt.
- Execution Phase (fresh sub-agent): The generated prompt runs in a clean context window, producing high-quality implementation on the first try.

What Makes This Effective

- The system consistently generates prompts with:
- XML structure for clear semantic organization
- Contextual "why" - explains purpose, audience, and goals
- Success criteria - specific, measurable outcomes
- Verification protocols - how to test that it worked
- "What to avoid and WHY" - prevents common mistakes with reasoning
- Extended thinking triggers - for complex tasks requiring deep analysis
- Trade-off analysis - considers multiple approaches and their implications

Most developers don't naturally think through all these dimensions. This system does, every time.

Ready



I have enough context to create the prompt. Ready to proceed?



Proceed

Create the prompt with current context



Ask more questions

I have more details to clarify



Let me add context

I want to provide additional information



Other

1 Submit answers

Esc to cancel

Ralph-loop

- Ralph is an autonomous AI agent loop that uses a bash script to run AI coding tools repeatedly until all tasks are complete.
- Requires clear exit criteria and max iteration count
- Each iteration is a fresh instance with clean context. Memory persists via git history, `progress.txt`, and `prd.json`.
- Multi-role ralph prompt



- Multi-role ralph prompt

```
PHASE 2 - ROTATING PERSONA REVIEW (cycle each iteration):
ITERATION MOD 6:

[0] CODE REVIEWER:
- Review code for bugs, security issues, edge cases
- Check error handling and types
- Fix any issues found

[1] SYSTEM ARCHITECT:
- Review file structure and dependencies
- Check separation of concerns
- Refactor if needed

[2] FRONTEND DESIGNER:
- Use /frontend-design skill
- Review UI/UX for this feature
- Improve components, accessibility, responsiveness

[3] QA ENGINEER:
- Run npm test
- Check test coverage, aim for 90%+
- Write missing unit tests for edge cases
- Run npm run lint && npm run build

[4] PROJECT MANAGER:
- Verify all acceptance criteria met
- Check .prompts/001-cloud-storage-import/spec.md requirements
- Document any gaps

[5] BUSINESS ANALYST:
- Review feature from user perspective
- Check if flows make sense
- Identify UX friction points

EACH ITERATION:
- Identify current persona (iteration % 6)
- Perform that persona's review
- Make ONE improvement or fix
- Commit with message: '[persona] description'
- If no issues found by ANY persona for 2 full cycles, output completion

OUTPUT <promise>FEATURE_READY</promise> when all personas report
" --completion-promise "FEATURE_READY" --max-iterations 30

```
```

# Reality Check

- AI code assistance acts as a kind of amplifier. If you're already doing the right things, AI will amplify the impact of those things.
- If you're already doing the wrong things, AI will help you to dig a deeper hole faster.
- Tools amplify capability. They don't replace it.

# Resources

- <https://github.com/anthropics/skills>
- <https://github.com/github/spec-kit>
- <https://github.com/glittercowboy/taches-cc-resources>
- <https://github.com/microsoft/autogen>
- <https://docs.crewai.com/en/introduction>
- <https://github.com/FoundationAgents/MetaGPT>
- Copilot Meta prompt workflow: <https://gist.github.com/carlokruger/067b073d6969b9bcf8580497d756f4fd>
- Claude Meta prompt workflow: <https://gist.github.com/carlokruger/067b073d6969b9bcf8580497d756f4fd>
- Multi-agent Ralph Loop: <https://gist.github.com/carlokruger/bc0afb5abe0d013270f2788236743688>